TP5 – Mini-projet

Système complet de recherche d'information

Département Informatique

3 avril 2026

1 Objectif général

Implémenter un **système complet de recherche d'information** capable d'indexer plusieurs types de documents (PDF, DOCX, TXT, HTML) et d'appliquer différents modèles de recherche pour répondre à des requêtes utilisateur.

Ce travail s'appuie sur les acquis des TP1 à TP4 et les enrichit avec :

— une **interface graphique** (Tkinter ou html),

— le **choix du modèle de recherche** (booléen classique, booléen étendu, flou, vectoriel),

— l'**affichage détaillé des résultats** (score, extrait du contenu, ouverture du document).

2 Travail à réaliser

2.1 Gestion des documents

Le système doit indexer des documents de différents types :

— **PDF** (via PyPDF2 ou pdfminer)

— **TXT** (fichiers texte bruts)

— **HTML** (fichiers .html, extraction du texte brut)

— **Docx**

Vous utiliserez **votre propre collection** de documents (au moins 40 documents, répartis dans les 4 types).

2.2 Indexation et prétraitement

1. Extraction du texte brut
2. Mise en minuscules
3. Suppression des mots vides (stopwords)
4. Calcul des poids **TF-IDF** pour chaque terme dans chaque document

Catégorie	Modèles
Booléen classique	ET, OU, NON (opérateurs ensemblistes)
Booléen étendu	p-norm (p=2)
Booléen flou	Lukasiewicz, Kraft (min-max)
Modèle vectoriel	Cosine, Dice, Jaccard, Overlap, Euclidienne, Produit scalaire

Table 1 – Modèles à implémenter

2.3 Modèles de recherche implémentés

2.4 Interface graphique

L'interface doit permettre à l'utilisateur de :

- Saisir une requête
- Choisir le **modèle de recherche**
- Lancer la recherche
- Afficher les résultats ordonnés par score décroissant
- Pour chaque résultat, afficher :
  - Nom du document
  - Score de pertinence
  - Deux lignes d'extrait contenant les termes de la requête
- Ouvrir directement le document depuis l'interface

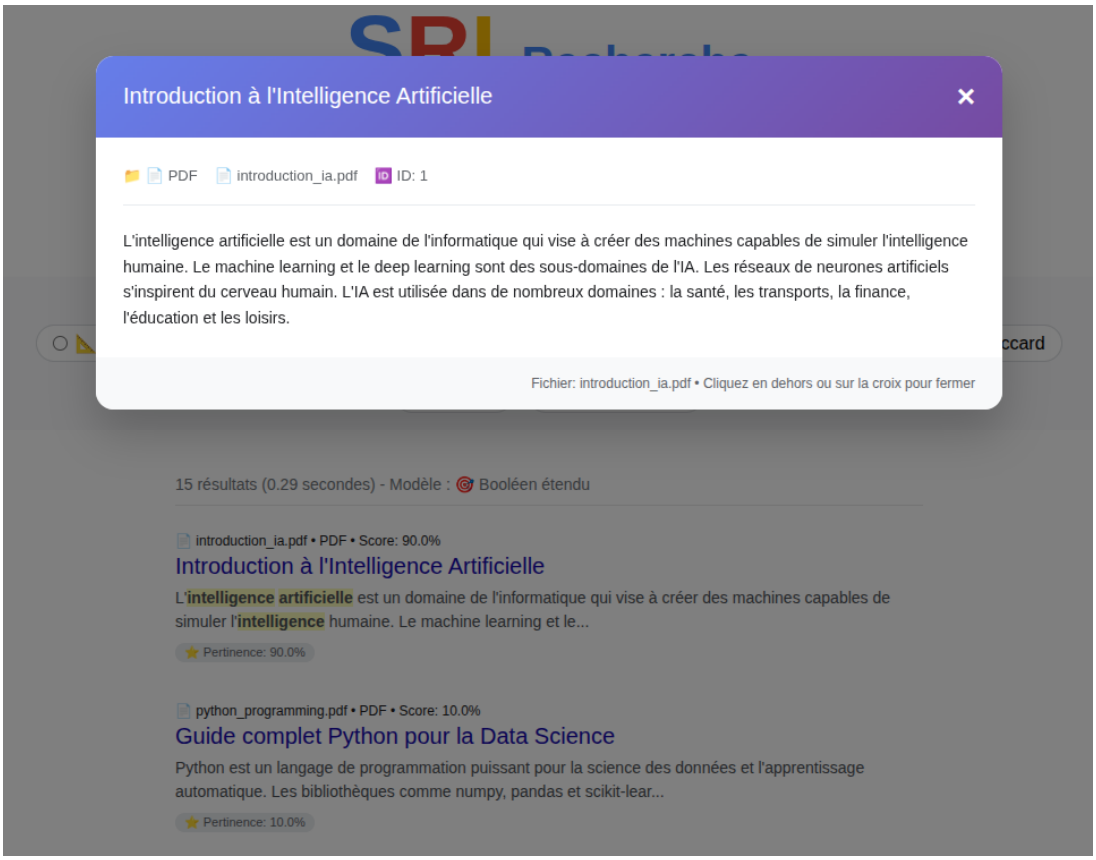
2.5 Modularité du code

```
TP5_MINI_PROJET/

          indexation/
                extracteur.py          # extraction texte PDF, TXT, HTML
                preprocesseur.py       # stopwords, stemming
                indexeur.py            # TF-IDF, index

          modeles/
                booleen_classique
                booleen_etendu
                flou_lukasiewicz
                flou_kraft
                vectoriel   # cos, jaccard, dice, overlap, euclidienne, produit scalaire

          interface/
                app.py                 # Tkinter

          collections/              # vos documents (PDF, DONX, TXT, HTML)

          main.py
          README.md
```

3 Exemple d'exécution

FIGURE 1 – Exemple d'exécution

L'utilisateur peut cliquer sur un résultat pour ouvrir le document.



4 Technologies autorisées

- Python 3.8+
- PyPDF2 ou pdfminer
- nltk (stopwords, stemmer)
- Tkinter (interface)
- os, glob, re, math, collections
- pandas (optionnel)

5 Rendu attendu

- Un dossier compressé (`TP5_NOM1_NOM2.zip`) contenant :
  - Tous les fichiers du projet
  - La collection de documents
  - Un `README.md` expliquant l'exécution, les modèles et la structure
- Un court rapport (2 pages max) présentant :
  - L'architecture du système
  - Une capture d'écran de l'interface
  - Une analyse comparative des modèles

6 Conseils et aide

— Testez chaque modèle indépendamment avant intégration.
— Réutilisez les fonctions des TP1 à TP4.

7 Question bonus (amélioration possible)

Proposez une fonctionnalité supplémentaire libre :

— Suggestions de requêtes
— Coloration des termes dans l'extrait
— Export des résultats
— Filtrage par type de document

Bon courage et bon développement !